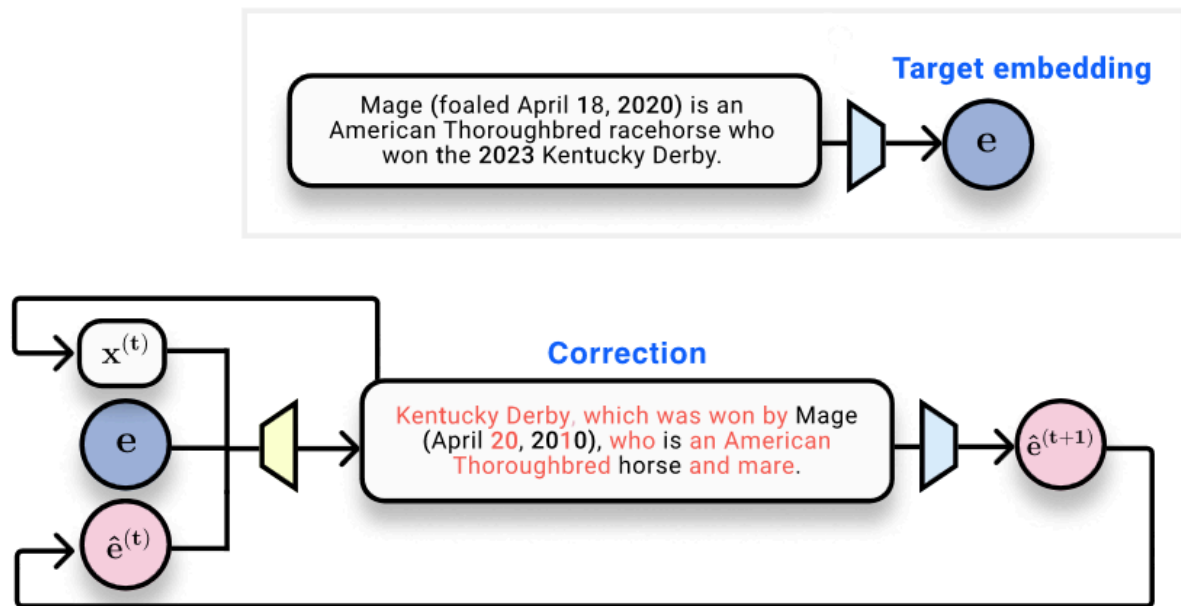




ABOUT ARTICLES NEWSLETTER PODCAST SUBSCRIBE WRITE
SUPPORT Q



Do text embeddings perfectly encode text?

05.MAR.2024 · 9 MIN READ

The rise of the vector database

As a result of the rapid advancement of generative AI in recent years, many companies are rushing to integrate AI into their businesses. One of the most common ways of doing this is to build AI systems that answer questions concerning information that can be found within a database of



Jack Morris

Jack Morris is a PhD student at Cornell Tech in New York City. He works on research at the intersection of

documents. Most solutions for such a problem are based on one key technique: Retrieval Augmented Generation (RAG).

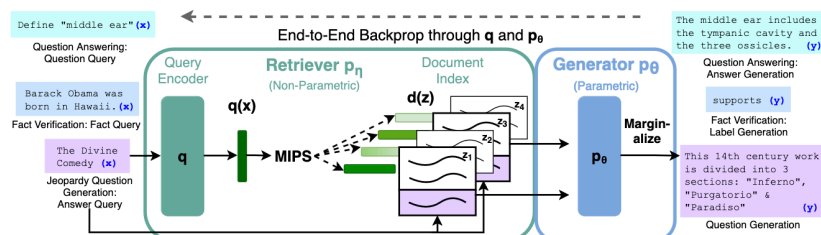


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder + Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

Overview of a RAG system. Source: “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”

This is what lots of people do now as a cheap and easy way to get started using AI: store lots of documents in a database, have the AI retrieve the most relevant documents for a given input, and then generate a response to the input that is informed by the retrieved documents.

These RAG systems determine document relevancy by using “embeddings”, vector representations of documents produced by an embedding model. These embeddings are supposed to represent some notion of similarity, so documents that are relevant for search will have high vector similarity in embedding space.

The prevalence of RAG has led to the rise of the *vector database*, a new type of database designed for **storing and searching through large numbers of embeddings**. Hundreds of millions of dollars of funding have been given out to startups that claim to facilitate RAG by making embedding search easy. And the effectiveness of RAG is the reason why lots of new applications are converting text to vectors and storing them in these vector databases.

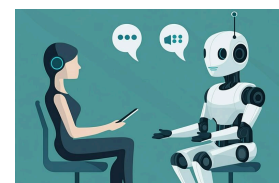
machine learning,
natural language
processing, and
security.

RECENT POSTS

1.

What's Missing From LLM Chatbots: A Sense of Purpose

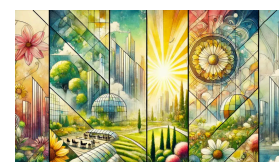
09.SEP.2024



2.

We Need Positive Visions for AI Grounded in Wellbeing

03.AUG.2024



3.

Financial Market Applications of LLMs

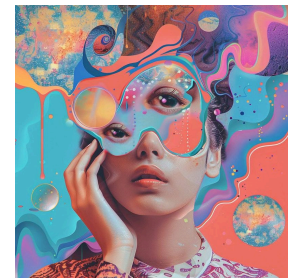
20.APR.2024



4.

A Brief Overview of Gender Bias in AI

08.APR.2024



5.
Mamba Explained
27.MAR.2024



| TAGS

Art

Conference

Deep
Learning

Ethics

Explainability

Generative
Models

Graphs

Healthcare

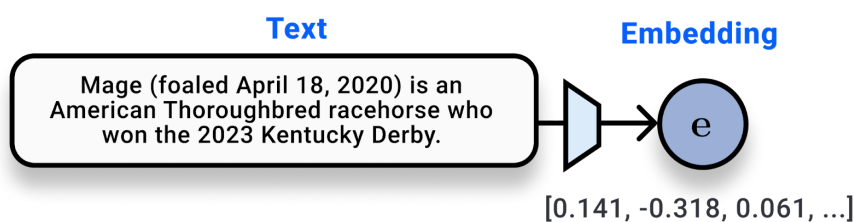
History

Human
factors

Impacts

Embeddings are hard to read

So what is stored in a text embedding? Beyond the requirement of semantic similarity, there are no constraints on which embedding must be assigned for a given text input. Numbers within embedding vectors can be *anything*, and vary based on their initialization. We can interpret the similarities of embedding with others but have no hope ever understanding the individual numbers of an embedding.



A neural embedding model (light blue) takes text input and produces an *embedding*, a vector that can be used for search.

Now imagine you're a software engineer building a RAG system for your company. You decide to store your vectors in a vector database. You notice that in a vector database, what's stored are embedding vectors, not the text data itself. The database fills up with rows and rows of random-seeming numbers that represent text data but never 'sees' any text data at all.

You know that the text corresponds to customer documents that are protected by your company's privacy policy. But you're not *really* sending text off-premises at any time; you only ever send embedding vectors, which look to you like random numbers.

What if someone hacks into the database and gains access to all your text embedding vectors – would this be bad? Or if the

service provider wanted to sell your data to advertisers – could they? Both scenarios involve being able to take embedding vectors and *invert* them somehow back to text.

From text to embeddings...back to text

The problem of recovering text from embeddings is exactly the scenario we tackle in our paper *Text Embeddings Reveal As Much as Text* (EMNLP 2023). Are embedding vectors a secure format for information storage and communication? Put simply: can input text be recovered from output embeddings?

Before diving into solutions, let's think about the problem a little bit more. Text embeddings are the output of neural networks, sequences of matrix multiplications joined by nonlinear function operations applied to input data. In traditional text processing neural networks, a string input is split into a number of token vectors, which repeatedly undergo nonlinear function operations. At the output layer of the model, tokens are averaged into a single embedding vector.

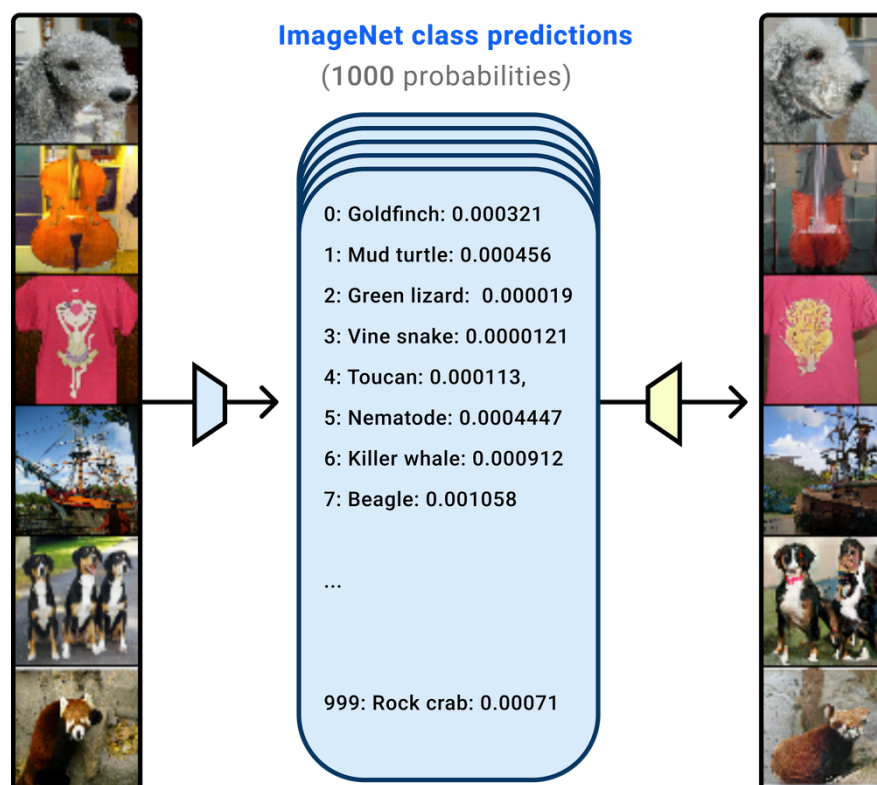
A maxim from the signal processing community known as the data processing inequality tells us that functions cannot add information to an input, they can only sustain or decrease the amount of information available. Even though conventional wisdom tells us that deeper layers of a neural network are constructing ever-higher-order representations, they aren't adding any information about the world that didn't come in on the input side.

[Interpretability](#)[Language](#)[LLM](#)[Machine Learning](#)[NLP](#)[Overviews](#)[Perspectives](#)[Podcast](#)[Policy](#)[Quantum ML](#)[Reinforcement Learning](#)[Science](#)[Speech Recognition](#)[Trends](#)[Vision](#)[XAI](#)

Additionally, the nonlinear layers certainly destroy *some* information. One ubiquitous nonlinear layer in modern neural networks is the “ReLU” function, which simply sets all negative inputs to zero. After applying ReLU throughout the many layers of a typical text embedding model, it is not possible to retain all the information from the input.

Inversion in other contexts

Similar questions about information content have been asked in the computer vision community. Several results have shown that deep representations (embeddings, essentially) from image models can be used to recover the input images with some degree of fidelity. An early result ([Dosovitskiy, 2016](#)), showed that images can be recovered from the feature outputs of deep convolutional neural networks (CNNs). Given the high-level feature representation from a CNN, they could invert it to produce a blurry-but-similar version of the original input image.



In computer vision, inversion models (yellow) have successfully reconstructed images given only the 1000 probability outputs of an ImageNet classifier, most of which are close to 0. (Images from *Understanding Invariance via Feedforward Inversion of Discriminatively Trained Classifiers*.)

People have improved on image embedding inversion process since 2016: models have been developed that do inversion with higher accuracy, and have been shown to work across more settings. Surprisingly, some work has shown that images can be inverted from the outputs of an ImageNet classifier (1000 class probabilities).

The journey to vec2text

If inversion is possible for image representations, then why can't it work for text? Let's consider a toy problem of recovering text embeddings. For our toy setting we'll restrict text inputs to 32 tokens (around 25 words, a sentence of decent length) and embed them all to vectors of 768 floating-point numbers. At 32-bit precision, these embeddings are $32 * 768 = 24,576$ bits or around 3 kilobytes.

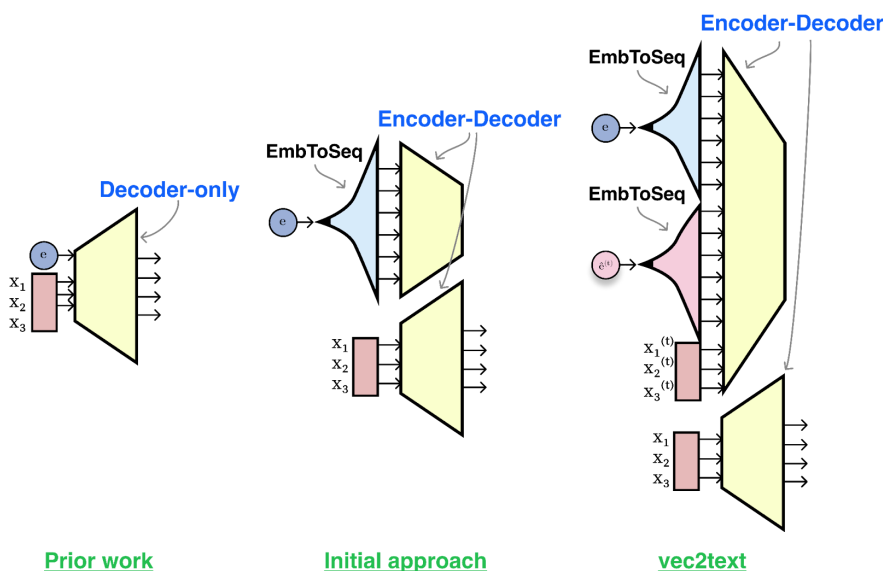
Few words represented by many bits. Do you think we could perfectly reconstruct the text within this scenario?

First things first: we need to define a measurement of *goodness*, to know how well we have accomplished our task. One obvious metric is "exact match", how often we get the exact input back after inversion. No prior inversion methods have any success on exact match, so it's quite an ambitious measurement. So maybe we want to start with a smooth measurement that measures how similar the inverted text is to the input. For this we'll use BLEU score, which you can just

think of as a percentage of how close the inverted text is to the input.

With our success metric defined, let us move on to proposing an approach to evaluate with said metric. For a first approach, we can pose inversion as a traditional machine learning problem, and we solve it the best way we know how: by gathering a large dataset of embedding-text pairs, and train a model to output the text given the embedding as input.

So this is what we did. We build a transformer that takes the embedding as input and train it using traditional language modeling on the output text. This first approach gives us a model with a BLEU score of around 30/100. Practically, the model can guess the topic of the input text, and get some of the words, but it loses their order and often gets most of them wrong. The exact match score is close to zero. It turns out that asking a model to reverse the output of another model in a single forward pass is quite hard (as are other complicated text generation tasks, like generating text in perfect sonnet form or satisfying multiple attributes).



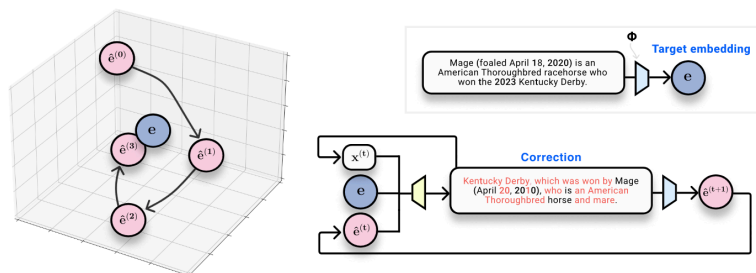
Overview of architectures considered. Prior work (left) uses a decoder-only architecture and inputs an embedding as a prefix.

We initially trained an encoder-decoder model (middle) to condition on an upscaled sentence embedding on the encoder-side. Our final method (right) includes an additional “hypothesis” text along with an upscaled hypothesis embedding.

After training our initial model, we noticed something interesting. A different way to measure model output quality is by re-embedding the generated text (we call this the “hypothesis”) and measuring this embedding’s similarity to the true embedding. When we do this with our model’s generations, we see a very high cosine similarity – around 0.97. This means that we’re able to generate text that’s close in embedding space, but not identical to, the ground-truth text.

(An aside: what if this weren’t the case? That is, what if the embedding assigned our incorrect hypothesis the same embedding as the original sequence. Our embedder would be *lossy*, mapping multiple inputs to the same output. If this were the case, then our problem would be hopeless, and we would have no way of distinguishing which of multiple possible sequences produced it. In practice, we never observe these types of collisions in our experiments.)

The observation that hypotheses have different embeddings to the ground truth inspires an optimization-like approach to embedding inversion. Given a ground-truth embedding (where we want to go), and a current hypothesis text and its embedding (where we are right now), we can train a corrector model that’s trained to output something that’s closer to the ground-truth than the hypothesis.



Overview of our method, Vec2Text. Given access to a target embedding e (blue) and query access to an embedding model ϕ (blue model), the system aims to iteratively generate (yellow model) hypotheses $\hat{e}$ (pink) to reach the target.

Our goal is now clear: we want to build a system that can take a ground-truth embedding, a hypothesis text sequence, and the hypothesis position in embedding space, and predict the true text sequence. We think of this as a type of ‘learned optimization’ where we’re taking steps in embedding space in the form of discrete sequences. This is the essence of our method, which we call vec2text.

After working through some details and training the model, this process works extremely well! A single forward pass of correction increases the BLEU score from 30 to 50. And one benefit of this model is that it can naturally be queried recursively. Given a current text and its embedding, we can run many steps of this optimization, iteratively generating hypotheses, re-embedding them, and feeding them back in as input to the model. With 50 steps and a few tricks, we can get back 92% of 32-token sequences exactly, and get to a BLEU score of 97! (Generally achieving BLEU score of 97 means we’re almost perfectly reconstructing every sentence, perhaps with a few punctuation marks misplaced here and there.)

Scaling and future work

The fact that text embeddings can be perfectly inverted raises many follow-up questions. For one, the text embedding vector contains a fixed number of bits; there must be some sequence length at which information can no longer be perfectly stored within this vector. Even though we can recover most texts of length 32, some embedding models can embed documents up to thousands of tokens. We leave it up to future work to analyze the relationship between text length, embedding size, and embedding invertibility.

Another open question is how to build systems that can defend against inversion. Is it possible to create models that can successfully embed text such that embeddings remain useful while obfuscating the text that created them?

Finally, we are excited to see how our method might apply to other modalities. The main idea behind `vec2text` (a sort of iterative optimization in embedding space) doesn't use any text-specific tricks. It's a method that iteratively recovers information contained in any fixed input, given black-box access to a model. It remains to be seen how these ideas might apply to inverting embeddings from other modalities as well as to approaches more general than embedding inversion.

To use our models to invert text embeddings, or to get started running embedding inversion experiments yourself, check out our Github repository:

<https://github.com/jxmorris12/vec2text>

References

Inverting Visual Representations with Convolutional Networks (2015), <https://arxiv.org/abs/1506.02753>

Understanding Invariance via Feedforward Inversion of Discriminatively Trained Classifiers (2021), <https://proceedings.mlr.press/v139/teterwak21a/teterwak21a.pdf>

Text Embeddings Reveal (Almost) As Much As Text (2023), <https://arxiv.org/abs/2310.06816>

Language Model Inversion (2024), <https://arxiv.org/abs/2311.13647>

Author Bio

Jack Morris is a PhD student at Cornell Tech in New York City. He works on research at the intersection of machine learning, natural language processing, and security. He's especially interested in the information content of deep neural representations like embeddings and classifier outputs.

Citation

For attribution in academic contexts or books, please cite this work as

```
Jack Morris, "Do text embeddings perfectly encode text?", The Gradient, 2024.
```

BibTeX citation:

```
@article{morris2024inversion,  
  author = {Jack Morris},  
  title = {Do text embeddings perfectly  
tly encode text?},  
  journal = {The Gradient},  
  year = {2024},  
  howpublished = {\url{https://thegr  
radient.pub/text-embedding-inversion}},  
}
```

Interpretability

LLM

NLP

You Might Be Interested In

| DEEP LEARNING

Modern AI is Domestification

27.MAY.2023 / TED XIAO

| DEEP LEARNING

In-Context Learning, In Context

29.APR.2023 / DANIEL BASHIR

| MACHINE LEARNING

Bootstrapping Labels via ____ Supervision & Human-In-The- Loop

05.MAR.2022 / EUGENE YAN

Tags

Art

Conference

Deep Learning

Ethics

Explainability

Generative Models

Graphs
Healthcare
History
Human Factors
Impacts
Interpretability
Language
LLM
Machine Learning
NLP
Overviews
Perspectives
Podcast
Policy
Quantum ML
Reinforcement Learning
Science
Speech Recognition
Trends
Vision
XAI